C++ Core Theory & Systems Revision Guide

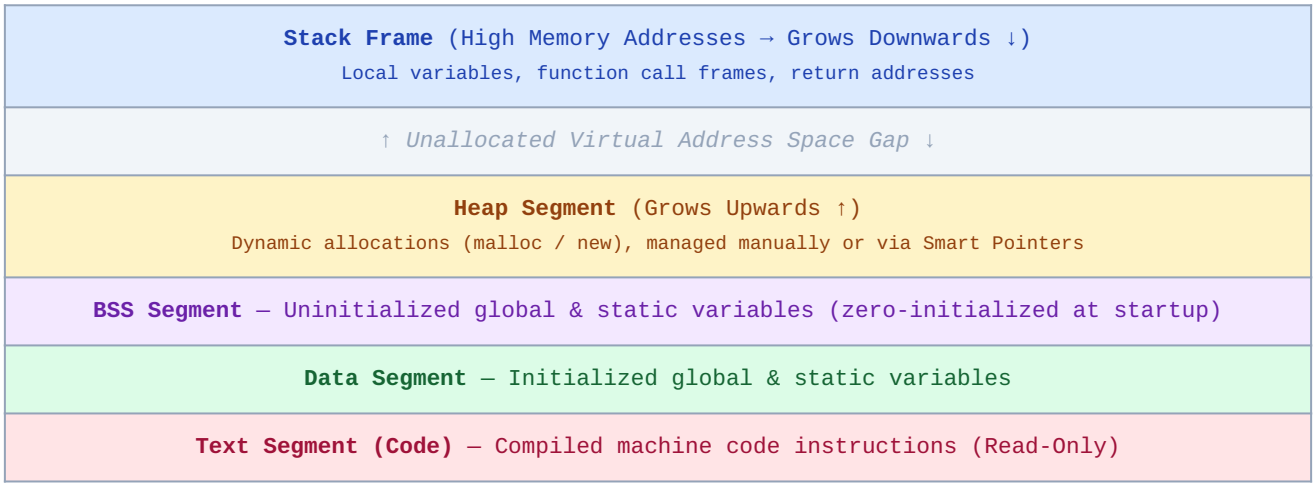
Comprehensive Reference for Memory Architecture, RAII, Cache Locality, STL Mechanics & Concurrency

Target: Low-Level C++ Mastery & Interview Preparation | **Focus:** Hardware-Aware Software Engineering | **Language Standard:** Modern C++ (C++11 through C++20/23)

Module 1: Process Memory Architecture & Stack vs. Heap

Understanding how an operating system structures virtual memory for a running executable is fundamental to writing performant C++ code.

1.1 Virtual Memory Layout of a C++ Process



1.2 Deep Dive: Stack vs. Heap Allocation

Attribute	Stack Memory	Heap Memory
Allocation Speed	**ULTRA FAST** (Single CPU pointer instruction modification)	**SLOWER** (Requires OS memory manager search & lock)
Lifetime	Deterministic, tied strictly to scope `{ }`	Dynamic, manually controlled until explicit `delete` or destructor call
Size Limit	Small (typically 1MB - 8MB per thread)	Large (bounded by physical RAM + Virtual Swap space)
Access Locality	**EXCELLENT** (Contiguous in CPU L1/L2 cache)	**VARIABLE** (Can cause fragmented cache misses)
Management	Automatic (Compiler pushes/pops stack frames)	Manual / RAII via Smart Pointers (`std::unique_ptr`, `std::shared_ptr`)

> **Stack Overflow Risk:** Allocating large raw arrays on the stack (e.g., `int arr[1000000];`) will exceed the fixed stack limit (usually 8MB) and trigger an immediate Segmentation Fault. Always use `std::vector` for dynamic or large datasets.

Module 2: Object Lifetime, RAII & Smart Pointers (No Garbage Collection)

Unlike Java, C#, or Python, C++ **does not use a Garbage Collector (GC)**. GC introduces non-deterministic execution pauses (stop-the-world), memory overhead, and lack of real-time guarantees. C++ uses **RAII** for zero-cost, deterministic lifetime management.

2.1 What is RAII (Resource Acquisition Is Initialization)?

RAII binds resource ownership (heap memory, file descriptors, sockets, mutex locks) to the **stack lifetime of an object**.

- **Acquire** resource in the *constructor* (e.g., allocate memory, open file, lock mutex).
- **Release** resource in the *destructor* (e.g., free memory, close file, unlock mutex).
- When the wrapper object leaves scope, its destructor automatically fires—even if exceptions are thrown!

2.2 Smart Pointer Architecture & Mechanics

Smart Pointer	Ownership Type	Internal Overhead	Key Use Case
`std::unique_ptr<T>`	Exclusive (1 owner)	**Zero Overhead** (Same size as raw pointer)	Default dynamic resource allocation. Non-copyable, move-only.
`std::shared_ptr<T>`	Shared (N owners)	Control Block (Reference Count + Weak Count) + Atomic increment/decrement	Multiple components sharing read/write ownership of a dynamic resource.
`std::weak_ptr<T>`	Observer (Non-owning)	Observes Control Block without incrementing Ref Count	Breaking cyclic references (e.g., Graph nodes, Parent-Child pointers).

2.3 Code Comparison: Manual vs. RAII Smart Pointers

```
// BAD: Manual Memory Management (Prone to leaks & exception bugs)
void legacyFunction() {
    Widget* w = new Widget();
    if (checkError()) return; // MEMORY LEAK! delete w was missed
    delete w;
}

// GOOD: RAII with std::unique_ptr (Modern C++ Standard)
void modernFunction() {
    auto w = std::make_unique<Widget>();
    if (checkError()) return; // Exception safe! Destructor automatically cleans up w
} // Cleaned up automatically on scope exit
```

2.4 Move Semantics (`T&&`) and `std::move`

Move semantics allow transferring raw dynamic memory resources from temporary objects (rvalues) without making deep expensive copies.

- **Lvalue:** Has an identifiable memory address (e.g., variable name `x`).
- **Rvalue:** Temporary value on the right-hand side with no persistent address (e.g., `x + 5`, return from function).
- `std::move(val)` casts an lvalue into an rvalue reference, allowing the move constructor to "steal" the underlying pointers.

Module 3: CPU Caching, Memory Locality & Alignment

Modern CPUs process billions of cycles per second, but fetching data from Main Memory (RAM) takes up to ~200 CPU cycles. To write hyper-fast code, you must design for the CPU Cache.

3.1 The Memory Latency Wall

Storage Medium	Approximate Access Latency	Capacity Range
CPU Registers	< 1 cycle (0.5 ns)	~1 KB
L1 Cache	~4 cycles (1 - 2 ns)	32 - 128 KB per core
L2 Cache	~12 cycles (3 - 5 ns)	512 KB - 1 MB per core
L3 Cache (Shared)	~40 cycles (10 - 20 ns)	16 - 128 MB shared
Main RAM	**~200 cycles (60 - 100 ns)**	16 - 128 GB

3.2 Cache Lines & Sequential Prefetching

Data is transferred from RAM into CPU Caches in fixed blocks called **Cache Lines (typically 64 bytes)**.

- **Cache Hit:** The required data address is already present in L1/L2/L3 cache. Execution completes in nanoseconds.
- **Cache Miss:** Data is missing from cache; the CPU stalls for ~200 cycles while fetching the 64-byte block from RAM.
- **Contiguous Data Locality:** Iterating over a contiguous array (`std::vector`) enables the CPU hardware prefetcher to load adjacent elements before they are requested.
- **Pointer Chasing Penalty:** Linked lists (`std::list`) and trees allocate nodes scattered randomly across heap memory, causing frequent cache misses.

3.3 Struct Padding & Alignment

CPUs read memory efficiently when primitive types are aligned to memory addresses divisible by their size (e.g., 4-byte `int` on a 4-byte boundary).

```
// Unoptimized Struct Layout: Size = 24 Bytes (due to implicit padding)
struct UnpaddedStruct {
    char   a; // 1 byte + 7 bytes padding
    double b; // 8 bytes
    int    c; // 4 bytes + 4 bytes padding
};

// Optimized Struct Layout (Reordered by member size): Size = 16 Bytes
struct OptimizedStruct {
    double b; // 8 bytes
    int    c; // 4 bytes
    char   a; // 1 byte + 3 bytes padding
};
```

Module 4: Standard Template Library (STL) Memory Profiles

Choosing the correct STL container is a memory architecture decision, not just an algorithmic one.

Container	Underlying Layout	Lookup / Access	Cache Locality	Memory Overhead
`std::vector<T>`	Single contiguous array block	$O(1)$ by index	MAXIMUM	Minimal (Cap - Size)
`std::deque<T>`	Array of fixed-size memory pages	$O(1)$ index	MODERATE	Page map pointers
`std::list<T>`	Doubly linked node pointers	$O(N)$ iteration	POOR (POINTER CHASING)	2 pointers per node (16B)
`std::unordered_map`	Hash table with linked list buckets	$O(1)$ average hash	POOR	Hash buckets + node pointers
`std::map`	Red-Black Tree (Balanced BST)	$O(\log N)$ tree traversal	POOR	3 pointers + color bit per node

4.1 Container Adapters: Stack & Queue mechanics

std::stack<T> (LIFO)

Adapter container. Uses `std::deque<T>` by default. Can be backed by `std::vector<T>` or `std::vector` for faster iteration cache.

```
std::stack<int, std::vector<int>> s;
```

std::queue<T> (FIFO)

Adapter container. Uses `std::deque<T>` by default. Cannot use `std::vector` because `pop()` requires $O(1)$ removal from front.

```
std::queue<int> q;
```

Module 5: Multithreading, Atomics & Memory Model

High-performance systems process workloads concurrently across multiple CPU cores.

5.1 Key Concurrency Hazards

- **Data Race:** Two or more threads concurrently access the same memory location where at least one thread modifies it without synchronization. Leads to undefined behavior!
- **Deadlock:** Two or more threads are blocked forever, each waiting for a mutex held by the other. Solved with `std::scoped_lock` (C++17 deadlock-free lock acquisition).
- **False Sharing:** Unrelated variables used by separate threads end up on the *same 64-byte cache line*. Cores repeatedly invalidate each other's L1 cache line, destroying speed.

5.2 Synchronization Primitive Hierarchy

```
// 1. Mutex Protection for Complex Operations
std::mutex mtx;
{
    std::lock_guard<std::mutex> lock(mtx); // RAII Mutex Locking
    sharedResource++;
}

// 2. Lock-Free Atomic Counter (Hardware Instruction level)
std::atomic<int> globalCounter{0};
globalCounter.fetch_add(1, std::memory_order_relaxed); // Fast lock-free operation
```

Module 6: Quick Revision Cheat Sheet & Rules of Thumb

Topic	Golden Rule / Key takeaway
Default Container	Always default to `std::vector` unless you explicitly need dynamic push-front ($O(1)$ queue requirement).
Resource Management	Never use raw `new`/`delete`. Wrap dynamic resources in `std::unique_ptr` or `std::shared_ptr`.
Pass-By-Value vs Reference	Pass small primitive types (≤ 16 bytes) by value; pass complex objects by `const T&`.
Cache Performance	Prefer arrays and contiguous vectors over pointer-based node linked lists to prevent CPU cache misses.
Rule of Five (C++11)	If you explicitly define a Destructor, Copy Constructor, or Copy Assignment, you must also define Move Constructor and Move Assignment.
Exception Safety	Ensure all dynamic locks and dynamic allocations use RAII so stack unwinding cleans resources on throw.